

# Logic for generating the random password

|                                                                                         |                                                                                                                  |
|-----------------------------------------------------------------------------------------|------------------------------------------------------------------------------------------------------------------|
| <input checked="" type="checkbox"/> Checkbox                                            |                                 |
|  EPICS |  <a href="#">Knowledge Base</a> |

## Generating the Random Password

Our app generates a **temporary random password** that the user will use during their first login.

To store this temporary password, we use the following data type:

```
private String password;
```

This variable will hold the randomly generated password.

## Creating the Random Password Method

We create a function whose job is to generate the password.

- Return type: `String`
- Parameter: `length` (the desired length of the password)

```
private String randomPassword(int length) {  
    String passwordSet = "ABCDEFGHJKLMNOPQRSTUVWXYZabcdefghijklmnopqrstuvwxyz01234567890!@#$%^&*()";  
}
```

Here, `passwordSet` is a **collection of allowed characters** from which the password will be generated.

## What We Need to Handle

To generate a random password, we must consider the following points:

1. The password must be of a specific length.
  2. `String` objects are **immutable** in Java.
  3. We need to repeatedly select random characters and add them together.
  4. Therefore, we need a **mutable** structure to build the password.
- 

## Why We Use `StringBuilder`

### The simple truth

We use `StringBuilder` because `String` cannot be modified.

That's the core reason. Everything else follows from this.

---

## What "String is immutable" Really Means

Consider the following code:

```
String s = "A";  
s = s + "B";
```

It may look like `s` is changed from `"A"` to `"AB"`, but here's what actually happens:

1. Java creates `"A"`.
2. Java creates a new String `"AB"`.
3. Java discards `"A"`.

Each modification creates a **new String object**.

---

## Why This is a Problem Inside a Loop

**✗** Using `String`

```
String password = "";

for (int i = 0; i < length; i++) {
    password = password + randomChar;
}
```

What happens here:

- The loop runs multiple times.
- A new String object is created every iteration.
- Older Strings are discarded.
- This leads to unnecessary memory usage and slower performance.

It works, but it's a bad practice.

---

## How **StringBuilder** Solves This

```
StringBuilder password = new StringBuilder();
password.append('A');
password.append('B');
```

Here:

- Only **one object** is created.
- Characters are added to the same object.
- Memory is reused efficiently.

This is what we mean by **mutable behavior**.

---

## Real-World Analogy

**String**

Writing on paper with ink.

- Want to change something?
- Rewrite the entire page.

### **StringBuilder**

Writing on a whiteboard.

- Add or remove characters freely.
- Same board, same space.

For repeated changes, Java prefers the whiteboard.

---

## **Why **StringBuilder** Is Perfect for Password Generation**

Your password generation logic:

- Uses a loop
- Appends characters repeatedly
- Works with dynamic lengths (8, 10, 12, etc.)

This is exactly the situation **StringBuilder** was designed for.

---

## **Converting Back to **String****

At the end of the method:

```
return password.toString();
```

Why?

- The outside world expects a **String**
- **String** is immutable and safe
- **StringBuilder** is only for internal construction

**Build with **StringBuilder** , return a **String** .**

---

## When NOT to Use `StringBuilder`

If you are doing simple concatenation:

```
String fullName = firstName + lastName;
```

This is perfectly fine. Java optimizes it internally.

## Rule to Remember

| If a String is modified inside a loop, use `StringBuilder`.

No exceptions for beginners.

## Complete Password Generation Logic

```
StringBuilder password = new StringBuilder();
Random random = new Random();

for (int i = 0; i < length; i++) {
    int index = random.nextInt(passwordSet.length());
    password.append(passwordSet.charAt(index));
}
```

## Understanding the Variables

- `i` → Counter that tracks how many characters have been added
- `length` → Desired password length
- `passwordSet.length()` → Total number of available characters

## What the `for` Loop Does

```
for (int i = 0; i < length; i++)
```

Meaning:

“Repeat this process length times.”

---

## What Happens in Each Iteration

### Step 1: Check the Condition

Example:

- `i = 5`
- `5 < 10` →  true

The loop continues.

---

### Step 2: Select a Random Index

```
int index = random.nextInt(passwordSet.length());
```

If:

- `passwordSet.length()` = 100

Then Java chooses a number from **0 to 99**, such as:

- 0 → first character
- 48 → middle character
- 99 → last character

All are equally likely.

---

### Step 3: Append the Character

```
password.append(passwordSet.charAt(index));
```

- The selected character is added to the password.
- Password length at this point = `i + 1`.

---

This explanation is now **clean, correct, beginner-friendly, and internally consistent**.

If someone reads this slowly, they will actually understand it instead of pretending to.